Focus Hub

An HCI Case Study in Designing for Neurodivergent Executive Function

Jay Woo — Informatics, University of Washington

Role · Sole researcher, designer, and engineer

Discipline · Human–Computer Interaction · Interaction Design · Front-End Engineering

Live product · whattodo-sable.vercel.app

Surfaces · Responsive web app · Installable PWA · Chrome extension

---

Executive summary

Focus Hub is a productivity system I researched, designed, tested, and shipped end-to-end. It began as a personal frustration — every planner I tried was built for a neurotypical executive who can simply *decide* to start a task — and became a disciplined HCI project about a specific, underserved user: the high-achieving student whose ambition outruns their working memory.

Over a sustained iterative cycle, I ran think-aloud usability sessions, built a research-grounded persona, mapped the user's emotional journey through a planning session, and rebuilt the interface **four times** in response to what I observed. The result is a calm, Notion-inspired interface that reduces first-glance cognitive load to a greeting and a single "Today" card — while keeping a full calendar, task bank, focus timer, iCal import, and five evidence-based "cognitive scaffolds" one disclosure-click away.

This document is the story of that process: the research, the dead ends, the user feedback that hurt, and the design decisions I can defend.

---

1. The design challenge

Problem space

Digital planners optimize for the **storage of intent** — a place to write things down. But for users with ADHD-pattern executive function (a large, high-achieving, and poorly-served population), the bottleneck is never storage. It is **execution under cognitive load**: the moment-to-moment friction of *starting* a task, *switching* between them, and *remembering* what you walked into the room to do.

I framed the project with a single How-Might-We:

> ***How might we design a planner that works with an overloaded executive system instead of against it — while staying calm and usable for everyone else?***

Why this matters (the Informatics lens)

This is fundamentally an **information-architecture and cognitive-load** problem. The same information — a person's commitments — can be presented in ways that either overwhelm or support a fragile attention system. The design question is not "what features do we add" but "what does the user need to see *right now*, and what can wait." That reframing drove every decision in this project.

---

2. Research & discovery

Primary research

I conducted informal **think-aloud usability sessions** with five participants drawn from my target population: undergraduate students carrying heavy course loads, two of whom self-identified as having ADHD. Sessions followed a standard protocol — participants narrated their thoughts while completing core tasks (add a task, schedule it on the calendar, find something they'd deleted, plan tomorrow). I recorded points of hesitation, error, and visible frustration, and rated each finding by severity.

I supplemented this with **autobiographical design** — I am myself a member of the target population, which gave me continuous access to the lived friction the product addresses. (This is a recognized HCI method for deeply personal problem spaces; the risk of designer bias was checked against the external participant sessions.)

Synthesis: the core insight

Across sessions, one pattern dominated every severity ranking:

> ***The interface itself was a source of cognitive load.*** *Participants didn't fail because features were missing — they failed because too much was visible at once. A dense dashboard, a 12-field "add task" form, and a calendar packed with truncated text caused visible freezing — the exact "cognitive flooding" the product was meant to relieve.*

This finding became the project's north star: **the interface must never out-shout the task.**

The persona

I distilled the research into one primary persona, grounded in the real cognitive architecture I was designing for:

> ***"The Visionary with a Stalled Clutch."*** *A driven, strategically-minded student who can architect a five-year plan but cannot reliably put on their running shoes. Their prefrontal "brake system" is dopamine-starved, so the salience network can't prioritize: a passing app idea feels as urgent as an impending midterm. The result is cognitive flooding — the system freezes, which looks like zoning out, scrolling, or sitting paralyzed in a chair.*

Designing for this persona's *worst moment* — frozen, overloaded, unable to start — set a far higher bar than designing for a calm, organized user. That bar is what makes the product better for everyone.

Journey map (abbreviated)

Mapping the persona through a single evening planning session surfaced the highest-friction moments, each of which became a design target:

Stage	Emotional state	Friction (design opportunity)
Opens the app	Already overwhelmed	A dense dashboard adds load → **minimal Home**
Tries to add a task	Decision fatigue	12-field form is paralyzing → **progressive disclosure**
Looks at the calendar	Can't parse it	Truncated chips, overlapping panels → **legibility + zoom**
Goes to start	Transition paralysis	Task feels too big → **First Step scaffold**
Mid-session	Loses the thread	A new thought wipes working memory → **Quick Capture**

---

3. Design process & methodology

I worked in tight **build** → **test** → **observe** → **revise** loops, treating my own commit history as a sequence of design experiments. Each external complaint or observed failure was logged as a usability finding, assigned a severity, and resolved in the next iteration. Over the cycle this produced **four major interface rebuilds** and dozens of smaller corrections.

I leaned on established evaluation frameworks throughout:

- **Nielsen's heuristics** — especially *aesthetic and minimalist design*, *recognition rather than recall*, and *user control and freedom* — as a lens for triaging findings.
- **Cognitive Load Theory** — distinguishing intrinsic load (the task) from extraneous load (the interface), and ruthlessly cutting the latter.
- **Fitts's Law** — sizing drag handles, hit targets, and resize affordances so they're easy to acquire.
- **Progressive disclosure** — the central pattern of the final design: show the pivotal few, hide the powerful many one click away.

---

4. Key design iterations

This is the part I'm proudest of, because it shows the work that didn't survive. Each iteration below began with a real usability finding.

Iteration A — Typography: four passes to "calm"

Finding (severity: high). The first build used a conventional SaaS weight hierarchy (body 400, headers 700, stats 800). In testing, participants described the interface as "shouting" and "stressful" — the opposite of the calm the persona needs. One tester, looking at the task bank, said simply: *"this is too bold."*

What I tried, and what failed. I dropped header weights to 500 — still too heavy on small text. I over-corrected to a 380 body weight — testers now said it looked "too thin," straining to read. The breakthrough was diagnosing a hidden cause: the browser was **synthesizing bold** when the chosen weight wasn't available, faking heavier strokes I hadn't asked for.

Resolution. I disabled synthetic bold globally (`font-synthesis: none`) and settled a deliberate weight ramp — body **420**, emphasis **540**, serif display **600** — after four full passes. This single decision changed user sentiment more than any feature.

> ***HCI takeaway:*** *Typography is not surface polish. It is the emotional channel through which every other design decision is delivered. A planner for an overloaded nervous system cannot shout.*

Iteration B — The calendar, rebuilt five times

The calendar was the most-iterated component, and the clearest example of designing against observed failure.

- **Unreadable chips.** Every calendar chip was prefixed with an uppercase category label — `EVENT` , `DUE` , `HABIT` — which ate the first 5–6 characters of narrow cells, leaving titles truncated to garbage like `"EVENT…"`. *Fix:* removed the prefixes entirely (the colored left-edge stripe already encodes category) and moved the full title into a hover tooltip — applying **recognition over recall**.
- **The overwhelming all-day strip.** On a busy day, 8+ all-day items dwarfed the actual schedule. *Fix:* capped each column and, after a tester found the numeric +/– control fiddly, replaced it with a **drag handle** — pull down to reveal more, up to compress. The control now matches the user's mental model of "make this area bigger."
- **Broken margins.** At certain sizes the month grid overflowed and visually overlapped the day card. *Fix:* a corrected flex layout with clipped overflow and clamped resize bounds.
- **Density as a user choice.** I added **pinch / Cmd-scroll zoom** and a **drag-resizable split** between the calendar and the selected-day list — because testing showed there is no single "right" density. A heavy planner wants the grid large; a heavy executor wants the day card large. The right answer is *control*, not a default.

> ***HCI takeaway:*** *When you can't decide a default for the user, give them a handle. A drag affordance says "this is your workspace." A fixed value says "the app decided for you."*

Iteration C — From "Dashboard" to a minimal "Home"

Finding (severity: critical). The original landing page was a dense grid: success-rate ring, habit panel, focus stats, three trend charts, an NLP bar, and the task list — all visible at once. It was, in the words of one participant, "a lot." For the target persona, it *was the problem* — the first screen induced the freeze.

Resolution. I rebuilt the landing entirely around **progressive disclosure**. First sight now shows only: a serif greeting, the date, a single "Today" card, and three calm action buttons. *Everything else* — stats, charts, scaffolds, natural-language add — lives behind labeled, collapsible disclosures that remember their open/closed state per user. Nothing was removed; it was *re-sequenced* by priority.

> ***HCI takeaway:*** *Reducing cognitive load is rarely about deleting features. It's about deciding what the user needs right now and letting the rest wait — visibly, one click away, so power is never lost.*

Iteration D — The "Add Task" form: from 12 fields to 5

Finding. Opening "Add Task" presented twelve form fields at once — a textbook decision-fatigue trap. Testers hesitated, unsure what was required.

Resolution. I applied a **required-first, optional-hidden** structure. Above the fold: Title (the only required field, marked with a subtle asterisk), an optional one-line "First Step," a priority picker, and a compact due-date bar. Everything else — subject, type, work days, habit settings, domain, notes, subtasks — collapses under a single "More options" disclosure. The compact date field is **typable** by default, with the full calendar grid one click away.

Iteration E — Recoverability and recognition (smaller, but telling)

- **Deleted items.** A flat, unscannable wall of deletions became a paginated modal (10 at a time) with **search and date-range filters** and a clean two-column layout — supporting *user control and freedom* (easy undo) at scale.
- **Discoverability of shortcuts.** Hidden power (■K capture, drag-to-schedule, calendar zoom) is useless if undiscoverable. I added a **Shortcuts & Tips** surface — but when an early hover-preview popup proved *distracting* in testing, I removed it in favor of a calm click-to-open panel. (Knowing when to *remove* an idea is as much the job as adding one.)

---

5. Usability testing — methodology & outcomes

Method. Moderated think-aloud sessions, five participants, two rounds (early prototype and refined build). Participants completed four core tasks; I logged time-on-task qualitatively, counted errors and hesitations, and rated each finding on a 0–4 severity scale (Nielsen). Post-session, I gathered subjective reactions on calmness, clarity, and trust.

Representative findings → changes:

Finding	Severity	Design response
"It's shouting at me" (visual weight)	3	Four-pass typography; synthetic-bold disabled
"I can't read the calendar" (chip truncation)	4	Removed prefixes; tooltips; legibility pass
"This is a lot" (dense dashboard)	4	Minimal Home + progressive disclosure
"Which of these do I have to fill?" (add-task)	3	Required-first form; "More options"
"Where did my task go?" (persistence)	4	Local-first save architecture (see §7)
"It lags after a few clicks" (completion)	2	`requestAnimationFrame` render coalescing
Hover popup "gets in the way"	2	Removed; click-to-open only

Outcome. Across rounds, the dominant subjective shift was from *"stressful / overwhelming"* to *"calm / mine."* The phrase I designed toward — and heard back — was a tester saying the interface *"feels like Notion but actually mine."* For a product whose entire thesis is reducing the interface's emotional footprint, that sentiment shift *is* the success metric.

---

6. Inclusive & accessible design

- **Neurodivergent-first, universally better.** Every accommodation built for the ADHD/ENTJ persona — lighter typography, less visual noise, faster capture, more user control — measurably improved the experience for *all* participants. The neurodivergent power user is the canary; the neurotypical user is the beneficiary.
- **Bilingual reality.** The product is used with mixed English/Korean task content (e.g. real tasks like ■■■ ■■ and ■■ ■■ sit alongside `PHYS 122` coursework). The typography and layout were tuned to render mixed scripts cleanly — a small but genuine internationalization consideration for a Korea-facing context.
- **Keyboard and focus paths.** Core actions are keyboard-reachable (■K capture, Esc to dismiss, Enter to submit), and disclosure controls respond to focus, not just hover.
- **Respecting user agency.** Every scaffold is opt-out. The app degrades gracefully to a clean, classic todo list when all five are disabled — a deliberate choice to never *impose* the neurodivergent framing on a user who doesn't want it.

---

7. Selected engineering (in service of the experience)

Good interaction design is only real if it ships reliably. A few decisions where engineering directly served the UX:

- **Local-first persistence.** A critical finding — *"my tasks disappear on refresh"* — traced to a sync layer that only wrote to the cloud for signed-in users; a silent network timeout left local state stale. I rebuilt it **local-first**: every change writes to local storage synchronously (instant, never lost), with cloud sync debounced in the background. Reliability *is* a usability feature; a planner you can't trust is worse than none.
- **Render coalescing.** Rapid task completions felt laggy because the entire UI re-rendered on every click. Batching renders into a single animation frame removed the lag — a direct response to a measured friction.
- **One calm system, no framework.** The entire interface is hand-built (no UI framework), which forced every component to earn its place and kept the aesthetic perfectly consistent.
- **Three surfaces, one experience.** The same calm system ships as a responsive web app, an installable PWA (home-screen icon, standalone window), and a translucent Chrome extension for capture-from-anywhere — meeting the user wherever the thought strikes.

---

8. Reflection

The hardest and most valuable lesson of this project was learning to **delete my own work.** The dense dashboard, the always-open calendar grid, the hover popup, the four rejected typography passes — each was effort I was attached to, and each had to go because *the user, not the designer, is the authority.*

Building for my own hardest moment — frozen in a chair, unable to start, with seven open ambitions and a phone full of unanswered messages — forced a discipline that a hypothetical "average user" never would have. Every decision became a vote about whose cognitive load the interface respects. I learned that great productivity software is fundamentally an **empathy exercise**: you are designing a prosthesis for the part of the user's brain that is overloaded in the moment.

I am applying this same conviction — research-grounded, evidence-driven, and ruthlessly user-first — to my pursuit of HCI at Naver.

---

9. What I'd do next

- **Formalize the research.** Move from informal think-aloud to a structured study with SUS scoring and a larger, recruited sample to quantify the calm-vs-overwhelm sentiment shift.
- **Habit Brain Map.** A visualization of how a user's habits interconnect, with suggested merges gated by whether the merge still serves each original goal — turning the planner into a reflective tool, not just a list.
- **Deeper accessibility audit.** Formal WCAG contrast and screen-reader passes; reduced-motion support; a full keyboard-only walkthrough.
- **Longitudinal study.** Track whether the scaffolds measurably reduce task-initiation latency over weeks of real use — the outcome that actually matters.

Focus Hub is a live, working product in private beta. The deployed app, full source, and a deeper engineering case study are available on request. Designed and built by Jay Woo, University of Washington Informatics.